UC Berkeley

Department of Electrical Engineering and Computer Sciences

EECS 127 - EECS 227AT. Optimization Models in Engineering

Lecture Note 25

Fall 2025

Instructor: Somayeh Sojoudi

Typeset by: George Ma

© EECS 127 - EECS 227AT. Optimization Models in Engineering

Matrix optimization appears a lot in machine learning and we study two problems here.

Matrix completion.

- Consider an unknown matrix $\mathbf{X}^* \in \mathbb{R}^{m\times n}$ and assume that we measure its entries X^*_{ij} only when (i,j) belongs to some given set $\mathcal{K}$. We know that $\mathbf{X}^*$ is low rank.
- The goal is to find $\mathbf{X}^*$ from the measured entries. Example:

$$\mathbf{X}^* = \begin{bmatrix} 1 & ? & 1 & ? & ? \\ ? & 2 & 3 & ? & 5 \\ ? & ? & 4 & 5 & ? \end{bmatrix}.$$

  In this case, $\mathcal{K} = \{(1,1),(1,3),(2,2),(2,3),(2,5),(3,3),(3,4)\}$, and ? are unknown entries.
- If $\mathbf{X}^*$ is full rank, then ?'s are arbitrary and we cannot uniquely find them.
- If $\mathbf{X}^*$ is rank 1 (or 2), then the unknown and known entries would be related to each other (for example, $\text{rank}(\mathbf{X}^*) = 1$ means the second and third rows in the above example are multiples of the first row).
- Therefore, the low-rank property gives us the possibility to learn $\mathbf{X}^*$ from a subset of its entries.

Example 1 (Netflix example)**.** Let $\mathbf{X}^* \in \mathbb{R}^{m\times n}$ where m is the number of people subscribed to Netflix, and n is the number of movies on Netflix. $X^*_{ij} \in \{1,2,3,4,5\}$ shows the rating of movie j by person i.

- Each person has seen only a small number of movies, so the full $\mathbf{X}^*$ is unknown.
- Netflix wants to learn $\mathbf{X}^*$ to find out the complete preferences of each person to suggest them the best list of movies customized to that person.
- $\mathbf{X}^*$ is low rank since many people have similar tastes.

■

Matrix completion can be written as

$$\begin{aligned} \min_{\mathbf{X}\in\mathbb{R}^{m\times n}} \quad & \text{rank}(\mathbf{X}), \\ \text{s.t.} \quad & X_{ij} = X^*_{ij},\ \forall (i,j) \in \mathcal{K}, \end{aligned} \tag{1}$$

where $\mathcal{K}$ is the given set of indices of known elements.

- Since we don't know the rank of $\mathbf{X}^*$, we search for the lowest-rank matrix $\mathbf{X}$ matching the data X^*_{ij}.
- Let $\sigma_1(\mathbf{X}),\ldots,\sigma_p(\mathbf{X})$ denote the singular values of $\mathbf{X}$, where $p = \min(m,n)$. Remember that $\text{rank}(\mathbf{X})$ is equal to the number of non-zero singular values, which is the same as the cardinality of $[\sigma_1(\mathbf{X}),\ldots,\sigma_p(\mathbf{X})]^{\mathrm{T}}$.

© EECS127/227AT. Optimization Models in Engineering, Somayeh Sojoudi, Fall 2025.

- Thus, (1) is equivalent to:

$$\begin{aligned} \min_{\mathbf{X}\in\mathbb{R}^{m\times n}} \quad & \left\| \begin{bmatrix} \sigma_1(\mathbf{X}) \\ \vdots \\ \sigma_p(\mathbf{X}) \end{bmatrix} \right\|_0, \\ \text{s.t.} \quad & X_{ij} = X^*_{ij},\ \forall (i,j) \in \mathcal{K}. \end{aligned} \tag{2}$$

- Recall that given $\boldsymbol{y} \in \mathbb{R}^p$, we have

$$\|\boldsymbol{y}\|_1 \leq \|\boldsymbol{y}\|_0 \text{ over the set } \{\boldsymbol{y} \mid |y_1|, \ldots, |y_p| \leq 1\}.$$

- Consider

$$\begin{aligned} \min_{\mathbf{X}\in\mathbb{R}^{m\times n}} \quad & \left\| \begin{bmatrix} \sigma_1(\mathbf{X}) \\ \vdots \\ \sigma_p(\mathbf{X}) \end{bmatrix} \right\|_1, \\ \text{s.t.} \quad & X_{ij} = X^*_{ij},\ \forall (i,j) \in \mathcal{K}. \end{aligned} \tag{3}$$

  (3) is a relaxation of (2) over the set

$$\left\{\mathbf{X} \mid \max\big(\sigma_1(\mathbf{X}), \ldots, \sigma_p(\mathbf{X})\big) \leq 1\right\} = \left\{\mathbf{X} \mid \|\mathbf{X}\|_2 \leq 1\right\},$$

  where the equality is by Theorem 1 in Lecture Note 9. (Note: since singular values are non-negative, $|\sigma_i(\mathbf{X})| \leq 1$ can be written equivalently as $\sigma_i(\mathbf{X}) \leq 1$.)

- (3) is equivalent to

$$\begin{aligned} \min_{\mathbf{X}\in\mathbb{R}^{m\times n}} \quad & \sigma_1(\mathbf{X}) + \cdots + \sigma_p(\mathbf{X}), \\ \text{s.t.} \quad & X_{ij} = X^*_{ij},\ \forall (i,j) \in \mathcal{K}. \end{aligned}$$

- The sum of singular values of a matrix is called *nuclear norm* and shown as $\|\mathbf{X}\|_*$.

- Thus, (3) is equivalent to

$$\begin{aligned} \min_{\mathbf{X}\in\mathbb{R}^{m\times n}} \quad & \|\mathbf{X}\|_*, \\ \text{s.t.} \quad & X_{ij} = X^*_{ij},\ \forall (i,j) \in \mathcal{K}. \end{aligned} \tag{4}$$

- Note: $\|\mathbf{X}\|_*$ is a norm, so it is convex. Thus, (4) is a convex relaxation of matrix completion (1) over the restricted space $\{\mathbf{X} \mid \|\mathbf{X}\|_2 \leq 1\}$, but outside of this space, it is not necessarily a relaxation.

- The above is similar to the compressed sensing literature but instead of finding a sparse vector, we aim to find a matrix $\mathbf{X}$ with a sparse set of singular values (i.e., many singular values should be zero).

- There are many conditions on $\mathbf{X}^*$ and k in the literature (beyond the scope of this course) showing (4) finds $\mathbf{X}^*$ correctly if the number of measurements is not too small.

© EECS127/227AT. Optimization Models in Engineering, Somayeh Sojoudi, Fall 2025.

Robust PCA.

- PCA was about low-rank approximation of a matrix $\mathbf{Y} \in \mathbb{R}^{m \times n}$.
- In robust PCA, we aim to decompose $\mathbf{Y}$ as the sum of two matrices $\mathbf{X} \in \mathbb{R}^{m \times n}$ and $\mathbf{Z} \in \mathbb{R}^{m \times n}$ such that $\mathbf{X}$ is low rank and $\mathbf{Z}$ is sparse (most entries should be zero).
- Given $\mathbf{Y}$, find $\mathbf{X}$ and $\mathbf{Z}$ such that

$$\mathbf{Y} = \mathbf{X} + \mathbf{Z}, \quad \text{where } \mathbf{X} \text{ is low rank, } \mathbf{Z} \text{ is sparse.}$$

- Define $\text{card}(\mathbf{Z})$ as the number of non-zero entries of $\mathbf{Z}$.
- To account for both rank and sparsity, we minimize a weighted sum:

$$\begin{aligned} \min_{\mathbf{X},\mathbf{Z} \in \mathbb{R}^{m \times n}} \quad & \text{rank}(\mathbf{X}) + \lambda \, \text{card}(\mathbf{Z}), \\ \text{s.t.} \quad & \mathbf{Y} = \mathbf{X} + \mathbf{Z}, \end{aligned}$$

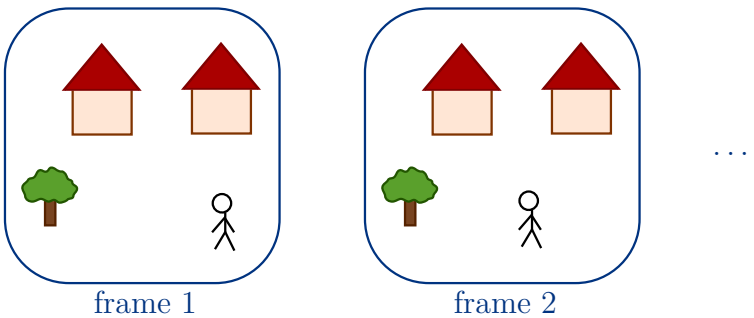
  where $\lambda > 0$ is a regularization coefficient.
- As before, we consider the convex model

$$\begin{aligned} \min_{\mathbf{X},\mathbf{Z} \in \mathbb{R}^{m \times n}} \quad & \|\mathbf{X}\|_* + \lambda \left(\sum_{i=1}^{m} \sum_{j=1}^{n} |Z_{ij}| \right), \\ \text{s.t.} \quad & \mathbf{Y} = \mathbf{X} + \mathbf{Z}. \end{aligned}$$

  (Here we don't use the notation $\|\mathbf{Z}\|_1$, as $\sum_i \sum_j |Z_{ij}|$ is not the same as matrix 1-norm.)
- This optimization serves as a convex relaxation over some restricted space.

Application in video processing.

- Consider the video of a place where the goal is to detect moving objects in that space.
- Example: A driverless car waiting behind a traffic light wants to detect moving pedestriants.
- A video is composed of frames:



© EECS127/227AT. Optimization Models in Engineering, Somayeh Sojoudi, Fall 2025.

- We decompose each frame as the sum of foreground and background.
- The background changes slowly across the frames (or does not change) while the foreground is the moving object(s).
- Assume we have 9 frames and we map them into matrices $\mathbf{U}^{(1)}, \ldots, \mathbf{U}^{(9)}$, where the (i,j)-th entry of matrix $\mathbf{U}^{(k)}$ $(k = 1, \ldots, 9)$ corresponds to the color value of the (i,j)-th pixel of frame k.
- Each $\mathbf{U}^{(k)}$ can be decomposed as $\mathbf{U}_{\text{b}}^{(k)} + \mathbf{U}_{\text{f}}^{(k)}$, where $\mathbf{U}_{\text{b}}^{(k)}$ corresponds to the background and $\mathbf{U}_{\text{f}}^{(k)}$ corresponds to the forebround.
- If the moving object(s) is a small part of the frame, then $\mathbf{U}_{\text{f}}^{(k)}$ is sparse.

  (If there are several moving objects, then the foreground dominates the background and the problem gets very hard.)
- Note that $\mathbf{U}_{\text{b}}^{(1)}, \ldots, \mathbf{U}_{\text{b}}^{(9)}$ are similar (almost identical) since the background changes slowly or is fixed, while $\mathbf{U}_{\text{f}}^{(1)}, \ldots, \mathbf{U}_{\text{f}}^{(9)}$ could be quite different due to moving object(s) across the frames.
- Define the function $\text{vec}(\cdot)$ that takes a matrix and reshapes it into a long vector by putting its columns on top of each other.

  Example:
$$\text{vec}\left(\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 3 \\ 2 \\ 4 \end{bmatrix}.$$
- Define $\mathbf{Y} = \begin{bmatrix} \text{vec}(\mathbf{U}^{(1)}) & \cdots & \text{vec}(\mathbf{U}^{(9)}) \end{bmatrix}$.
- Note that $\mathbf{Y}$ is given (since the frames are known).
- We want to decompose $\mathbf{Y}$ as:
$$\mathbf{Y} = \mathbf{X} + \mathbf{Z}, \quad \text{where } \mathbf{X} \text{ is low rank, } \mathbf{Z} \text{ is sparse.}$$

  In fact, $\mathbf{X}$ plays the role of $\begin{bmatrix} \text{vec}(\mathbf{U}_{\text{b}}^{(1)}) & \cdots & \text{vec}(\mathbf{U}_{\text{b}}^{(9)}) \end{bmatrix}$, and $\mathbf{Z}$ plays the role of $\begin{bmatrix} \text{vec}(\mathbf{U}_{\text{f}}^{(1)}) & \cdots & \text{vec}(\mathbf{U}_{\text{f}}^{(9)}) \end{bmatrix}$.
- Since $\mathbf{U}_{\text{b}}^{(1)}, \ldots, \mathbf{U}_{\text{b}}^{(9)}$ are similar, $\mathbf{X}$ is low rank; and since moving object is a small part of the frames, $\mathbf{Z}$ is sparse.
- We solve robust PCA:
$$\begin{aligned} \min_{\mathbf{X},\mathbf{Z}} \quad & \|\mathbf{X}\|_* + \lambda \left(\sum_{i=1}^{m} \sum_{j=1}^{n} |Z_{ij}| \right), \\ \text{s.t.} \quad & \mathbf{Y} = \mathbf{X} + \mathbf{Z}. \end{aligned}$$

© EECS127/227AT. Optimization Models in Engineering, Somayeh Sojoudi, Fall 2025.

- There are strong theorems in the literature showing this idea works. Check Example 13.10 for pictures.

Numerical algorithms.

- We previously showed that the least-squares $\min_{\boldsymbol{x}} \|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|$ has a closed-form solution, e.g., $\boldsymbol{x}^* = \mathbf{A}^+\boldsymbol{b}$.
- This is an exception and almost all optimization problems have no formula for their solution.
- Consider $\min_{\boldsymbol{x}\in\mathbb{R}^n} f(\boldsymbol{x})$ where we can't find the solution in one shot (i.e., via a formula), so we should find it gradually.
- Say we generate a sequence of points:

  $$\boldsymbol{x}^{(0)} \to \boldsymbol{x}^{(1)} \to \boldsymbol{x}^{(2)} \to \cdots,$$

  where $\boldsymbol{x}^{(0)}$ is our initial guess for $\boldsymbol{x}^*$.
- We work on $\boldsymbol{x}^{(0)}$ and improve it to find a better guess $\boldsymbol{x}^{(1)}$. Then we work on $\boldsymbol{x}^{(1)}$ to improve it to $\boldsymbol{x}^{(2)}$, and this continues.
- This is called an *iterative method*.
- Can we design the sequence in a way that $\boldsymbol{x}^{(k)} = \boldsymbol{x}^*$ for some $k \in \{0, 1, 2, \dots\}$ so that we can stop in finite time?
- Example: $f(x) = (2 - x^2)^2 \Rightarrow x^* = \sqrt{2} \approx 1.414$. The solution is irrational and it has infinitely many digits. If we use a computer, it has finite precision and can't generate and store a number with infinitely many digits.
- So, finding $\boldsymbol{x}^*$ is not a reasonable task and we aim to find an approximate solution.
- Given a precision error $\varepsilon > 0$ (ε can't be zero), find an approximate solution $\hat{\boldsymbol{x}}$ such that

  $$\|\boldsymbol{x}^* - \hat{\boldsymbol{x}}\| \le \varepsilon.$$

- Going back to $\min f(\boldsymbol{x})$, we generate a sequence $\boldsymbol{x}^{(0)} \to \boldsymbol{x}^{(1)} \to \boldsymbol{x}^{(2)} \to \cdots$ and stop at time k if we are sure that $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| \le \varepsilon$.

  Note: We don't know $\boldsymbol{x}^*$, so we can't calculate $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|$ and need an alternative formula to certify that $\boldsymbol{x}^{(k)}$ is at most ε away from $\boldsymbol{x}^*$.
- To generate $\boldsymbol{x}^{(0)} \to \boldsymbol{x}^{(1)} \to \cdots$, we need an algorithm that tells us how to obtain $\boldsymbol{x}^{(k+1)}$ from $\boldsymbol{x}^{(k)}$ for $k = 0, 1, \dots$.
- An algorithm is called a *descent algorithm* if it generates the sequence in such a way that

  $$f(\boldsymbol{x}^{(k+1)}) < f(\boldsymbol{x}^{(k)}), \quad \forall k \in \{0, 1, \dots\}.$$

© EECS127/227AT. Optimization Models in Engineering, Somayeh Sojoudi, Fall 2025.

- We write $\boldsymbol{x}^{k+1}$ as $\boldsymbol{x}^{(k)} + s^{(k)}\Delta\boldsymbol{x}^{(k)}$ where $s^{(k)} \geq 0$ is a scalar and $\Delta\boldsymbol{x}^{(k)} \in \mathbb{R}^n$.
- $s^{(k)}$ is called *step size* and $\Delta\boldsymbol{x}^{(k)}$ is called *direction*.
- The equation $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + s^{(k)}\Delta\boldsymbol{x}^{(k)}$ is called *updating rule*.

© EECS127/227AT. Optimization Models in Engineering, Somayeh Sojoudi, Fall 2025.